画像処理実験 第 7 回

学生番号: 09B24490
加藤弘都 (KATO, HIROTO)

2026 年 10 月 23 日

1 課題 7

1.1 [7-2]

1.1.1 calcScore 関数

```
  1:   for i in range(len(w)):   # ** ここから 3 行は「for x,y,u,v in w:」でも良い
  2:     x=w[i,0]; y=w[i,1]
  3:     u=w[i,2]; v=w[i,3]
  4:     x_=H[0,0]*x+H[0,1]*y+H[0,2]
  5:     y_=H[1,0]*x+H[1,1]*y+H[1,2]
  6:     z_=H[2,0]*x+H[2,1]*y+H[2,2]
  7:     du=u-x_/z_
  8:     dv=v-y_/z_ # (du,dv) は，(x,y) を H で変換した座標 と (u,v) の差．
  9:     if du*du+dv*dv<10: score+=1 # w[i] が正しい場合の処理 (※ 1 閾値)
 10:     else:              pass # w[i] が正しくない場合の処理
 11:   return score
```

calcScore 関数は，w の u，v と射影変換で計算した u，v を比較している．

$$\begin{pmatrix} u_i \\ v_i \end{pmatrix} = \frac{1}{z'_i}\begin{pmatrix} x'_i \\ y'_i \end{pmatrix} \text{ ただし } \begin{pmatrix} x'_i \\ y'_i \\ z'_i \end{pmatrix} = H\begin{pmatrix} x_i \\ y_i \\ 1 \end{pmatrix} \tag{1}$$

まず x，y について getH 関数で作成した変換行列 H を用いて，上記の射影変換をして u，v を計算する．次に先程の x，y に対応する u，v（w に保存されてるもの）との差 du，dv を計算する．この二乗和が 10 より小さいとき，score を 1 増やす．最後に score を返している．

1.1.2 getH 関数

```
  1:     if bestScore < score:                    # TODO: 最大値を残すようにする
  2:         bestScore=score       # 多数の試行で得られる score の最大値が bestScore
  3:         bestH=H               # 現在までに見つかった最良の H が bestH
```

getH 関数は trial 回だけ 4 個の対応点をランダムに選択して一番正確な H を見つける．w のうち 4 個の対応点を選択するたびにの calcScore 関数の返り値（正確だった回数）が bestScore より上回ったときに新しい bestScore と H を保存する．そして，trial 回選んだ後の H を返している．

1.1.3 画像合成

このとき，変換行列は以下のようになる．

```
1: best matrix:
2:  [[  0.80021062   0.05643845 254.80984697]
3:  [ -0.12207815   0.99858559  26.36415685]
4:  [ -0.00032581   0.00011106   1.        ]]
```

この変換行列を用いて，画像を合成すると図 1 のようになる．この画像を見ると，正確に特徴点を対応させて画像を合成できている．そのため，誤った特徴点の対応が含まれなかったと考える．



図 1　合成画像

1.1.4　自分の画像合成

これまでに作成したコードを用いて，自分の画像を合成すると図 2 のようになる．合成の精度を高くするために，getH 関数で 4 つの対応点を選択する回数の trial=600 や特徴座標 w の数 max=100 などのように変更している．getH 関数ではランダムで選択するため，trial を増やすことで誤った H を作る可能性があるからである．また，max を増やすことでどんな場合でも正確だった点の数を多くすることができると考えた．

図 2 をみると，2 つの画像を正確に合成できていることがわかる．またコードを複数回実行したとき，trial，max の変更前は score が 1 下がっただけで合成に失敗することが多かった．しかし，変更後は score が小さくなっても score は 30 より小さくならず，合成に大きく失敗することが減った．

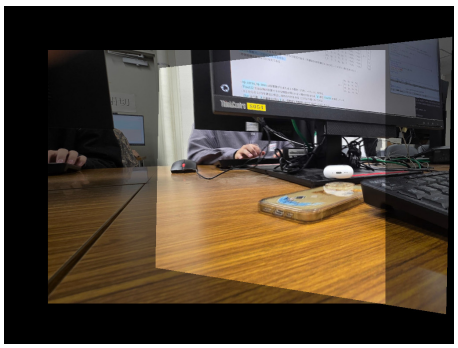


図 2　自分の 2 枚の画像の合成画像

1.1.5　H の変更による合成の精度改善

calcScore 関数で見つけた正しい対応点のみを参考に H を作成することで精度の高い H を生成する．

まず，calcScore 関数で正しい対応点を見つけるたびにリスト right_w に格納して，リストを関数の返り値とする．次に，getH 関数ではリストを rightW に一時的に保存しておき，以下のように最大の score

を更新するときに bestW に格納する．

```
1:      if bestScore < score:                   # TODO: 最大値を残すようにする
2:          bestScore=score         # 多数の試行で得られる score の最大値が bestScore
3:          bestH=H                 # 現在までに見つかった最良の H が bestH
4:          bestW=right_w
```

ループ終了後，リスト bestW に対応点が 8 こ以上保存されているなら，calcHomography(bestW) で bestH を更新する．これにより，正しい対応点 bestW のみを使って H を計算できる．

このときの画像は図**??**のようになる．図を見ると，比較的正確な画像ができていることがわかる．

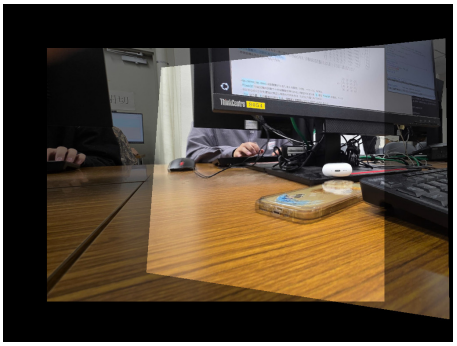


図 3　合成画像

2　感想

自動で正しい点をたくさん選択できたらもっといいのになーと感じた．

参考文献